Le système DNS (Domain Name System)

1. Fonctionnement du système DNS

Le **système DNS** (*Domain Name System*) a en charge d'établir la correspondance entre un nom pleinement qualifié (FQDN) et une adresse IP. Le système DNS permet à des hôtes du réseau de soumettre des requêtes à un serveur DNS afin d'obtenir **l'adresse IP** d'un hôte connaissant le **nom** de cet hôte (par exemple www.google.com → 209.85.229.99). Cette traduction des noms en adresses IP doit toujours être réalisée puisque que **seule l'adresse IP permet de communiquer** sur le réseau.

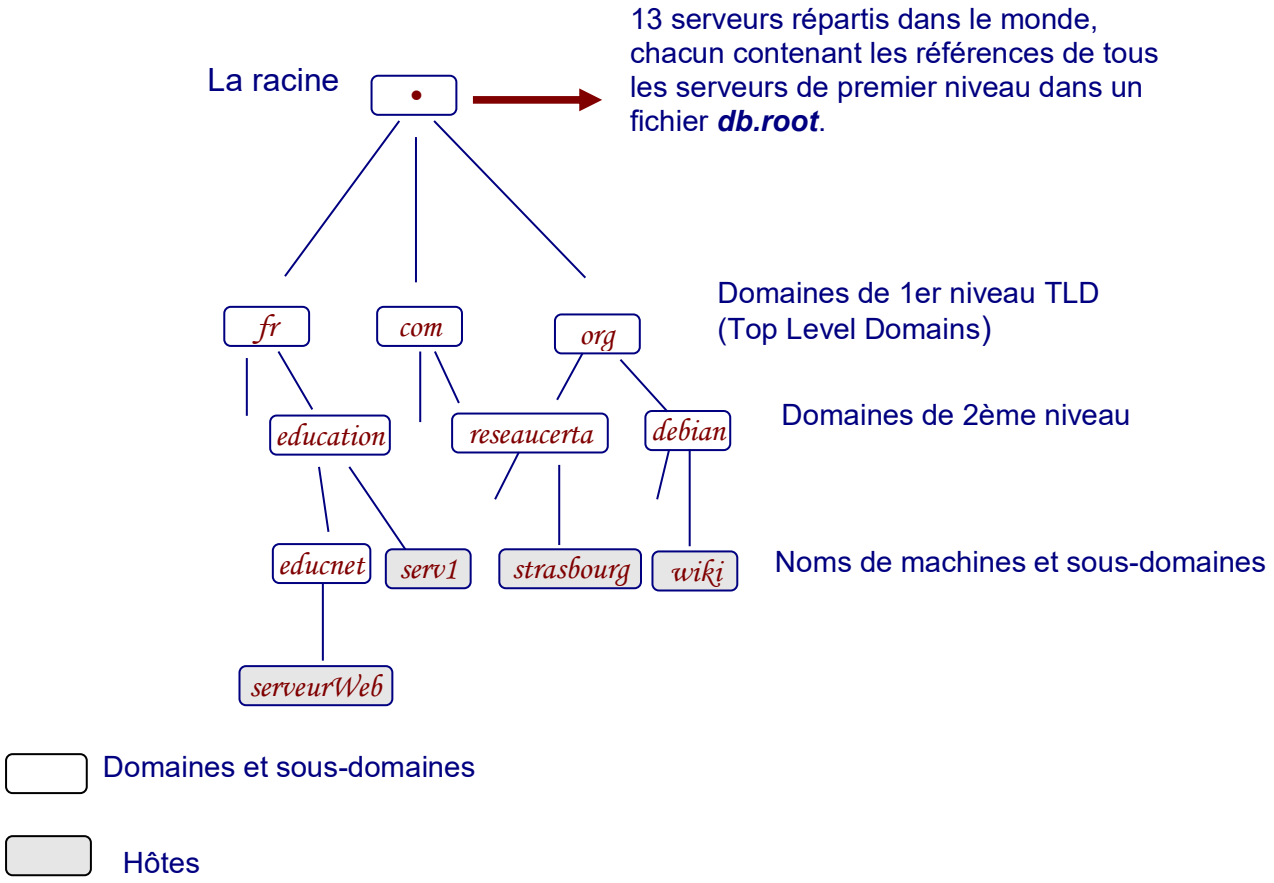
Il s'agit d'un modèle en **arborescence hiérarchique** avec une **gestion décentralisée des données** (chacun étant responsable des données de sa zone).

Le système de noms DNS se présente sous forme d'un **arbre inversé** avec pour sommet "**la racine (.)**" et un ensemble de domaines dits de **premier niveau** (.fr, .org, .com, .edu, .net, etc.). Viennent après les domaines de **2ème niveau**, de **3ème niveau**, etc.

Un serveur de noms particulier s'occupe d'un nœud de l'arborescence ou d'un ensemble de nœuds sur lequel il aura **autorité**. On dit que le serveur gère une **zone d'autorité.** C'est à dire qu'il gérera l'attribution des noms et résoudra les noms via **une base de données** s(matérialisée par ce qu'on appelle un **fichier de zone**) distincte pour chaque nœud. Chaque information élémentaire de la base de données DNS est un objet appelé "*resource record*" (RR).

Un nœud peut contenir aussi bien des domaines que des noms de machines.

Le système DNS peut se schématiser ainsi :



Le nom pleinement qualifié

a. Quel est le nom pleinement qualifié du serveur « serveurWeb » qui figure sur le schéma ci-dessus ?

serveurWeb.educnet.education.fr

b. Dans l'exemple ci-dessus, quel est le serveur DNS va résoudre le nom d'hôte pleinement qualifié du poste « serveurWeb » ?

C'est le serveur qui gère la zone educnet (serveur autoritaire de la zone)

Le serveur racine ne sait pas où se trouve cet hôte, par contre il possède un enregistrement pour le **domaine "fr".**

La **zone "fr"** est aussi restreinte au nœud correspondant. Son fichier de zone inclut donc des informations sur les délégations de gestion du reste du domaine dont le domaine "education".

Le fichier de zone "educnet.education.fr" possède l'information concernant l'hôte "serveurWeb" et pourra ainsi résoudre le nom **serveurWeb.educnet.education.fr.**

Pour de raisons de performance et de sécurité, il est conseillé de configurer les serveurs de noms pour qu'ils n'acceptent les requêtes en mode récursif que pour les machines de la zone pour laquelle ils sont autoritaires.

La zone in-addr.arpa (zone reverse)
Le domaine in-addr.arpa est un domaine spécial chargé de réaliser les recherches inversées, c'est-à-dire retrouver un **nom** en connaissant **l'adresse IP**.

Par exemple, la zone de recherche inverse pour le réseau **192.168.1.0/24** dans le domaine sera **1.168.192.in-addr.arpa.** Cette zone devra répondre pour toutes les adresses déclarées dans la tranche **192.168.1.1 à 192.168.1.254.** On inscrira dans cette zone tous les nœuds du réseau pour lesquels on désire que la résolution inverse fonctionne.

Des exemples de fichiers d'enregistrements de zone sont donnés en annexe 2.

2. Mécanisme de résolution de noms sur Internet

La résolution de noms est le mécanisme essentiel qui permet à un client DNS d'obtenir la liste des adresses IP associées à un nom DNS. Si on ne tient pas compte des mécanismes de cache et des serveurs DNS transitaires, le principe de base en est le suivant :
Un client DNS doit être configuré pour connaître l'adresse IP d'au moins un serveur DNS directement accessible.

- Le client DNS envoie une demande de résolution de nom DNS, en utilisant le protocole de transport **UDP**, vers le port bien connu **53** du premier serveur DNS de son fichier de configuration (`/etc/resolv.conf`). Le client attend ensuite la réponse, pendant un certain délai. S'il n'a pas reçu de réponse à l'expiration du délai, il peut interroger le serveur DNS suivant, s'il y en a un déclaré dans son fichier de configuration.
- Le serveur DNS traite la demande :
    - S'il a autorité sur le domaine demandé, il consulte sa base de noms et répond **directement au client**.

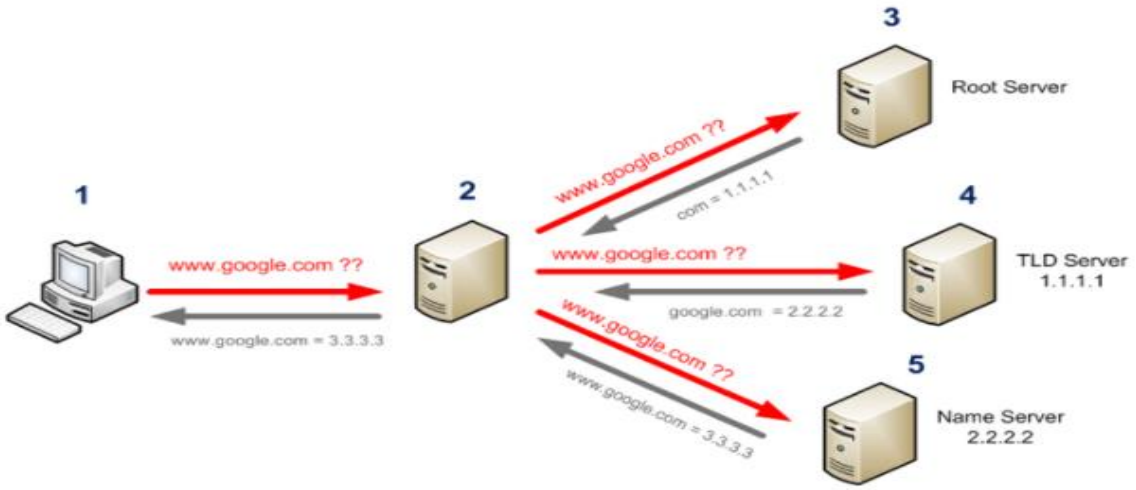
- S'il n'a pas autorité, le serveur DNS effectue une recherche **récursive** : il interroge l'un des serveurs de la **racine** DNS, qui va lui retourner la liste des serveurs de noms du domaine de **premier niveau** correspondant au nom recherché.

Le serveur va ensuite réitérer la demande, de niveau en niveau, jusqu'à trouver le serveur DNS ayant autorité sur le sous-domaine recherché, qui lui retournera la liste des adresses correspondant au nom FQDN.

- Finalement, le serveur DNS retourne la réponse à son client.

Quand un serveur DNS a obtenu une réponse à sa demande, il la **stocke** dans son **cache**, pour pouvoir l'utiliser ultérieurement s'il reçoit la même demande. La durée de validité d'une réponse mise en cache est configurée par un paramètre associé à l'enregistrement de la relation nom de domaine/adresse, appelé TTL (*Time To Live*).

Exemple : Demande de résolution du nom DNS www.google.com :



1	Le client envoie au résolveur une requête DNS demandant l'adresse IP correspondant au nom www.google.com.
2	Le résolveur envoie la requête au serveur racine demandant l'adresse IP correspondant au serveur de 1er niveau .com
3	Le serveur racine envoie l'adresse IP du serveur .com
4	Le résolveur envoie la requête au serveur .com demandant l'adresse IP correspondant au serveur google
5	Le serveur .com envoie l'adresse IP du serveur google
6	Le résolveur envoie la requête au serveur google demandant l'adresse IP correspondant au serveur www
7	Le serveur google envoie l'adresse IP du serveur www
8	Le résolveur envoie l'adresse IP de www au client

Annexe 1 : les fichiers de configuration de bind9

Fichier named.conf.default-zones

```
zone "." {
      type hint;
      file "/usr/share/dns/root.hints";
};
```

"hint" est le type pour le fichier des serveurs racines, qui ont autorité pour la zone racine point (".").

Chemin vers le fichier dans lequel sont listés les différents serveurs root avec leurs adresses IP.

```
zone "localhost" {
        type master;
        file "/etc/bind/db.local";
};

zone "127.in-addr.arpa" {
        type master;
        file "/etc/bind/db.127";
};

zone "0.in-addr.arpa" {
        type master;
        file "/etc/bind/db.0";
};

zone "255.in-addr.arpa" {
        type master;
        file "/etc/bind/db.255";
};
```

Déclaration de la zone "localhost" pour laquelle le serveur a autorité :
"**type master**" car serveur maître sur la zone)
"**file**" donne le chemin vers le fichier de zone correspondant (donné ici en chemin absolu)

Toute déclaration de zone suit ce modèle.

Déclaration des zones reverses imposées par la RFC1912

Fichier named.conf.local : déclarations de zones primaire et secondaire.

Exemple pour une zone primaire (master) :

```
zone "exemple.fr" {
      type master;
      file "db.exemple.fr";
};
```

Chemin donné en relatif car ce fichier sera créé dans le répertoire de stockage défini par la directive "directory" du fichier **named.conf.options.**
Remarque : si le chemin relatif ne fonctionne pas, utiliser le chemin **absolu**.

Exemple pour une zone secondaire (slave) :

```
zone "exemple.fr" {
      type slave;
      file "slave/db.exemple.fr";
      masters {192.1.1.1;};
};
```

Adresse du serveur maître.
Voir explications en **annexe 4**

Exemple pour une zone inverse maître :

```
zone "1.192.168.in-addr.arpa" {
      type master;
      file "db.192.168.1.rev";
};
```

Fichier named.conf.options

```
options {
        directory "/var/cache/bind";
      …
};
```

Annexe 2 : les fichiers de zone

Le fichier db.local

```
; BIND data file for local loopback interface
$TTL    604800

@       IN      SOA     localhost. root.localhost. (
                              2          ; Serial
                         604800          ; Refresh
                          86400          ; Retry
                        2419200          ; Expire
                           3600 )        ; Negative Cache TTL
;
@       IN      NS      localhost.
@       IN      A       127.0.0.1
@       IN      AAAA    ::1
```

Durée de vie en secondes par défaut d'un enregistrement de ressource (RR) : ici 7 jours

4 informations utiles pour le serveur secondaire (voir **annexe 4**)

Durée de présence d'une réponse négative dans les caches suite à une question sur le domaine

Chaque information élémentaire du fichier de zone est un objet appelé "*resource record*" (RR).

Numéro de série (*serial*) : Identifie la version de la zone ; quand on modifie le fichier de zone, on incrémente ce numéro. Le format conseillé est le suivant : YYYYMMDDxx.

Rafraîchissement (*refresh*) : intervalle en secondes destiné au serveur secondaire pour rafraîchir son fichier de zone (nombre décimal entier sur 8 chiffres).

Tentatives (*retry*) : intervalle en secondes avant de recontacter le serveur principal en cas d'échec de la demande de rafraîchissement.

Expiration (expire) : indique le temps en secondes, au bout duquel un serveur secondaire doit éliminer toutes les informations de zone s'il n'a pas pu contacter le serveur (cette valeur doit être élevée).

Toutes les "*refresh*" secondes, le serveur secondaire transfère le SOA de la zone et vérifie si le numéro de série a augmenté ; si c'est le cas le transfert de zone a lieu. En cas d'échec de cette interrogation, le serveur secondaire recommence toutes les "*retry*" secondes jusqu'à atteindre le temps d'expiration (*expire*).

Les types d'enregistrement :

L'enregistrement de type SOA (**S**tart **O**f **A**uthority : responsable de la zone) est obligatoirement le premier : il donne les caractéristiques techniques générales de la zone. Il est suivi de l'adresse mail de l'administrateur qui ne comporte pas de signe « @ » (remplacé par un point).

L'enregistrement de type NS définit un serveur de nom primaire ou secondaire pour la zone DNS.

L'enregistrement de type A met en correspondance un nom de machine et une adresse IPv4.

L'enregistrement de type AAAA met en correspondance un nom de machine et une adresse IPv6.

L'enregistrement de type CNAME définit un **alias** pour un nom existant : plusieurs noms pointent sur la même adresse IP.

L'enregistrement de type MX indique le nom du serveur de messagerie du domaine avec une priorité.

Le fichier db.127

```
; BIND reverse data file for local loopback interface

$TTL    604800
@       IN      SOA     localhost. root.localhost. (
                              1         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      localhost.
1.0.0   IN      PTR     localhost.
```

L'enregistrement PTR permet de faire de la résolution inverse c'est-à-dire associer à une adresse IP un nom FQDN.

Le fichier db.exemple.fr

```
$TTL  86400
@                IN    SOA   ns1.exemple.fr. hostmaster.exemple.fr. (
                             2008113001  ;serial
                             86400       ;refresh
                             21600       ;retry
                             3600000     ;expire
                             3600 ) ;negative caching ttl
@                IN    NS          ns1.exemple.fr.
@                IN    NS          ns2.exemple.fr.
exemple.fr.      IN    MX          servmail

servmail         IN    A           192.168.1.200
ns1              IN    A           192.168.1.50
ns2              IN    A           192.168.1.100
servftp          IN    A           192.168.1.150
mail             IN    CNAME       servmail.exemple.fr.
ftp              IN    CNAME       servftp
```

L'enregistrement MX définit un serveur de mail. Attention, un MX ne peut pas être référencé par un alias.

L'enregistrement CNAME indique que le nom est un alias vers un nom existant.

Le fichier db.192.168.1.rev

```
$TTL  86400
@                IN    SOA   ns1.exemple.fr. hostmaster.exemple.fr. (
                             2008113001  ;serial
                             86400       ;refresh
                             21600       ;retry
                             3600000     ;expire
                             3600 ) ;negative caching ttl
```

```
@                IN    NS          ns1.exemple.fr.
@                IN    NS          ns2.exemple.fr.

200              IN    PTR         servmail.exemple.fr.
50               IN    PTR         ns1.exemple.fr.
100              IN    PTR         ns2.exemple.fr.
150              IN    PTR         servftp.exemple.fr.
```

A savoir : La délégation de zone

La délégation de zone transfère la responsabilité d'un sous-domaine à un autre serveur de noms.

La délégation de zone est "déclarée" dans le fichier de zone du domaine parent par un enregistrement de type NS.

Et un type d'enregistrement A est ensuite nécessaire pour la correspondance entre l'adresse IP et le nom.

Exemple :

```
$TTL  86400
@                     IN    SOA   ns1.exemple.fr.
                                  hostmaster.exemple.fr. (
                                  ... )
@                                 IN    NS          ns1.exemple.fr.
...

delegation                        IN    NS    d1.delegation.exemple.fr.
d1.delegation.exemple.fr.         IN    A     192.168.1.30
```

Annexe 3 : Le serveur secondaire

Un serveur de noms secondaire ne gère pas directement les informations sur les zones mais les obtient à partir du serveur de noms principal de la zone (ou d'un autre serveur secondaire) via le réseau (**transfert de zone**). Un serveur secondaire ne peut modifier des données de la zone mais il a lui aussi autorité sur la zone.

Cette redondance permet une meilleure tolérance aux pannes et une réduction de la charge de travail des serveurs principaux.

Concrètement, un serveur secondaire peut être mis à jour de deux façons :

- en fonction de la valeur '*refresh*' définie dans le SOA de la zone ;
- ou lorsqu'il reçoit une notification du primaire : actif par défaut (l'option "*notify*" est par défaut à "*yes*") sur bind9.

Quand il démarre, un serveur secondaire doit connaître son **serveur maître** pour entamer un transfert de zone avec ce serveur.

Dans le fichier **named.conf.local du serveur secondaire :**

```
zone "exemple.fr" {
      type slave;
      file "slave/db.exemple.fr";
      masters { 192.1.1.1; };
};
```

Il est préférable de créer un répertoire que l'on nommera "slave" dans le répertoire d'accueil des fichiers de zone pour les zones secondaires pour deux raisons :

- cela identifie clairement les zones secondaires, d'autant plus qu'un serveur peut être à la fois maître sur certaines zones et esclave sur d'autres.
- Cela réduit les problèmes de sécurité car comme le démon named fonctionne sous l'autorité de l'utilisateur bind et le groupe bind, il faut rendre le répertoire dans lequel seront stockés les fichiers de zone secondaires accessible en écriture ; cela limite donc au minimum la portée de ces droits.

Sur le serveur principal, il est préférable de limiter le transfert de zone au(x) seul(s) serveur(s) secondaire(s).

```
zone "exemple.fr" {
      type master;
      file "db.exemple.fr";
      allow-transfer { 192.168.1.100; };
};
```

Puis, il faut **ajouter dans chaque fichier de zone à exporter (donc sur le primaire)** un **enregistrement "NS" pour chaque serveur esclave** (l'option activée par défaut "*notify yes*" agit d'ailleurs uniquement sur l'adresse IP renseignée dans ces enregistrements).

Lorsque le serveur secondaire démarre pour la première fois, **il crée automatiquement son fichier de zone à partir du serveur primaire**. Attention, il faut que le groupe système "bind" ait le droit d'écrire dans le répertoire "/var/cache/bind/slave"... ce qui n'est certainement pas le cas puisque vous n'avez pu le créer que sous l'utilisateur "root". Dans ce cas : `chmod g-w /var/cache/bind` (les fichiers de la zone secondaire étant dans "slave", on peut enlever, pour ce répertoire, ce droit au groupe bind)

```
chgrp bind /var/cache/bind/slave
chmod g+w /var/cache/bind/slave
```

db.exemple.fr

```
$TTL  86400
exemple.fr.  IN   SOA   ns1.exemple.fr.  hostmaster.exemple.fr.  (

                  2008113001  ;serial
                  86400       ;refresh
                  21600       ;retry
                  3600000     ;expire
                  3600 ; negative caching ttl )

@                       IN    NS          servSec.exemple.fr.
servSec                 IN    A           192.168.1.100

...
```

Numéro de série (*serial*) : Identifie la version de la zone ; quand on modifie le fichier de zone, on incrémente ce numéro. Le format conseillé est le suivant : YYYYMMDDxx.

Rafraîchissement (*refresh*) : intervalle en secondes destiné au serveur secondaire pour rafraîchir son fichier de zone (nombre décimal entier sur 8 chiffres). Cette valeur peut être élevée si on a maintenu l'option "*notify yes*" au niveau du serveur maître.

Tentatives (*retry*) : intervalle en secondes avant de recontacter le serveur principal en cas d'échec de la demande de rafraîchissement.

Expiration (expire) : indique le temps en secondes, au bout duquel un serveur secondaire doit éliminer toutes les informations de zone s'il n'a pas pu contacter le serveur (cette valeur doit être élevée).

Toutes les "*refresh*" secondes, le serveur secondaire transfère le SOA de la zone et vérifie si le numéro de série a augmenté ; si c'est le cas le transfert de zone a lieu. En cas d'échec de cette interrogation, le serveur secondaire recommence toutes les "*retry*" secondes jusqu'à atteindre le temps d'expiration (*expire*).

Mais si l'on a maintenu, sur le serveur primaire, l'option par défaut de notification, le serveur maître notifiera immédiatement tout changement de son fichier de zone au serveur secondaire.

Attention : à chaque modification du fichier de zone, il ne faut pas oublier d'augmenter le numéro de série et de faire une relecture par le démon du fichier de configuration.

Annexe 4 – Tester un serveur de noms

Les outils dont on dispose :

Le paquet dnsutils fournit quelques commandes utiles :

- Pour vérifier les fichiers de configuration globale : **`named-checkconf`**

Si la commande ne renvoie rien, c'est qu'il n'y a pas d'erreurs de syntaxe dans les fichiers.

- Pour vérifier la syntaxe des fichiers de zone : **`named-checkzone zonename filename`**

  Il ne faut pas toujours se fier au "OK" final mais lire les phrases qui précèdent...

- **Les commandes** `nslookup`, `host` **et** `dig` permettent de vérifier les données relatives à chaque zone.

Mais pour ne pas perdre de temps, il est important de procéder méthodiquement.

Première batterie de tests : le serveur bind 9 a-t'il correctement démarré ou redémarré ?

ATTENTION, SAUF s'il y a une erreur de syntaxe dans les fichiers de configuration globaux, le serveur va apparemment démarrer ou redémarrer sans erreur. Pour voir si tout c'est réellement bien passer, il faut lire les fichiers de trace (*logs*).

Lorsque vous lancez ou relancez le serveur, il est judicieux d'ouvrir une autre console, d'y lancer la commande "tail -f /var/log/syslog" pour lire en direct les fichiers de trace concernant le démon *named*.

Voici par exemple 2 cas où il y a un problème :

Premier cas : le fichier de zone n'est pas trouvé :

```
Jan 26 16:01:33 ns1 named[21056]: zone domaine-grp1.com/IN: loading from master
file db.domaine-grp1.com failed: file not found
```

Deuxième cas : le fichier de zone est trouvé mais présente une erreur :

```
Jan 26 16:13:57 ns1 named[21652]: zone domaine-grp1.com/IN: loading from master
file /etc/bind/db.domaine-grp1.com failed: unexpected end of input
```

Une erreur se situe ici dans le fichier de configuration de la zone.

On a le détail des erreurs avec la commande :

```
named-checkzone zonename filename /var/cache/bind/db.domaine-grp1.com
```

Erreurs courantes :

- les nombreux ";" des fichiers de configuration globale ;
- les noms DNS pleinement qualifiés dans les fichiers de zone non suffixés par un point ;
- les parenthèses mal placées de l'enregistrement SOA ;
- La non-relecture de bind9 lorsqu'un fichier de configuration est modifié et le non vidage du cache.

Deuxième batterie de tests : vérification des données relatives à chaque zone.

L'utilisation des commandes `nslookup`, `host` ou `dig` vous permettra de vérifier les données relatives à chaque zone.

Nous ne détaillerons ici que la commande dig (**D**omain **I**nformation **G**roper) qui donne beaucoup de détails... À l'inverse de `nslookup`, cette commande n'est pas interactive.

```
dig www.reseaucerta.org
```

```
; <<>> DiG 9.5.0-P2 <<>> www.reseaucerta.org
;; global options:  printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 26702
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 3, ADDITIONAL: 3
```

La requête a réussi

flag "ra" : le serveur est récursif

Nombre de réponses pour chaque section

```
;; QUESTION SECTION: (1)
;www.reseaucerta.org.            IN      A

;; ANSWER SECTION: (2)
www.reseaucerta.org.     1785    IN      CNAME   strasbourg.reseaucerta.org.
strasbourg.reseaucerta.org.  3585 IN     A       130.79.130.89

;; AUTHORITY SECTION: (3)
reseaucerta.org.         10785   IN      NS      c.dns.gandi.net.
reseaucerta.org.         10785   IN      NS      b.dns.gandi.net.
reseaucerta.org.         10785   IN      NS      a.dns.gandi.net.

;; ADDITIONAL SECTION: (4)
a.dns.gandi.net.         117683  IN      A       217.70.179.40
b.dns.gandi.net.         117683  IN      A       217.70.184.40
c.dns.gandi.net.         117683  IN      A       217.70.182.20

;; Query time: 2 msec (5)
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Wed Jan 28 12:20:18 2009
;; MSG SIZE  rcvd: 187
```

L'adresse IP du serveur DNS qui a répondu

(1) La section QUESTION reprend la requête émise.

(2) La section ANSWER donne la réponse à la requête.

(3) La section AUTHORITY donne les serveurs de noms ayant autorité sur la zone.

(4) La section ADDITIONAL donne les adresses IP des serveurs de noms autoritaires.

(5) La section Query time donne le temps de réponse de la requête. Cette valeur indique donc si la réponse est en cache ou pas.

Pour filtrer les résultats, on peut préciser, dans la requête, les différents types de RR :
dig www.reseaucerta.org NS
dig www.reseaucerta.org A
dig www.reseaucerta.org MX
dig -x 192.168.1.124
Etc.

L'option +trace de la commande dig permet de faire la recherche en parcourant l'arborescence depuis la racine jusqu'à la réponse.

```
dig +trace www.domaine-grp1.com

; <<>> DiG 9.5.0-P2 <<>> +trace www.domaine-grp1.com
;; global options:  printcmd
.                       3600000 IN      NS      A.ROOT-SERVERS.NET.
;; Received 48 bytes from 192.168.1.2#53(192.168.1.2) in 0 ms

domaine-grp1.com.       3600    IN      NS      ns.domaine-grp1.com.
;; Received 72 bytes from 192.168.1.124#53(A.ROOT-SERVERS.NET) in 14 ms

www.domaine-grp1.com.  86400 IN      CNAME      ns.domaine-grp1.com.
ns1.domaine-grp1.com.  86400 IN      A         192.168.10.10
domaine-grp1.com.      86400 IN      NS         ns.domaine-grp1.com.
;; Received 72 bytes from 192.168.10.10#53(ns.domaine1-grp1.com) in 3 ms
```

Remarque : attention au cache qui pourrait fournir des réponses qui ne sont plus valables ; la commande `rndc flush` permet de supprimer toutes données en cache.